

Privacy Boost Protocol Extensions Audit



September 1, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	4
Scope	5
Updates	6
System Overview	7
Audited Changes	7
Security Model and Trust Assumptions	9
Chain and Token Assumptions	9
Off-Chain Components	9
Privacy Boundary	11
Privileged Roles	11
Automated Soundness Analysis	12
Critical Severity	14
C-01 EdDSA Signature Verification Is Satisfiable With a Forged Signature	14
Medium Severity	15
M-01 Fixed Gateway Gas Reserve Does Not Scale With Remaining Slots and Can Revert an Entire Epoch	15
Low Severity	16
L-01 Disallowed ERC-20 Recipient Can Revert Entire Portal Deposit Epoch	16
L-02 Auth Key Mutations Fail for Account Owners that Delegate Under EIP-7702	17
L-03 Secret-Bearer Claims Are Not Constrained to Public-Payout Settlement	18
L-04 An EOA Delegating to PortalDelegate Under EIP-7702 Can No Longer Receive ETH	18
L-05 Forced-Withdrawal Fees Stranded upon Treasury Unset	19
L-06 Fee-Note Ciphertext Metadata Is Not Integrity-Protected	20
Notes & Additional Information	21
N-01 Poseidon Hash Function Accepts Non-Canonical Arity	21
N-02 Non-Canonical Signature Scalar $S = \ell$ Accepted	21
N-03 Portal Note Randomness Lacks Domain Separation by Deployment	22
N-04 Forced Withdrawal Recipient Public Input Is Not Range-Constrained	22
N-05 AuthContext Field Layout Is Split Across Two Layers, and the Circuit Reads as Though Half of It Were Padding	23
N-06 Missing Security Contact	24
N-07 Use calldata Instead of memory	24
N-08 Unnecessary unchecked Blocks	25
N-09 Gift Claim Circuit Inactive Slot Constraints are Unreachable	25
N-10 cancelDeposit Does Not Follow CEI Pattern	26

N-11 Non-Descriptive Names	26
N-12 Magic Numbers in the Code	28
N-13 Inconsistent Integer Base Within Contracts	29
N-14 Missing or Incomplete Docstrings	30
Client Reported	30
CR-01 Gift Refunds Authorize Safe-Owned Spends on an Operator-Held Key	30
CR-02 Direct-Key Gift Refunds Do Not Authenticate the Payout Destination	31
CR-03 A Withdrawal Marker Can Be Reopened as a Normal Note	32
CR-04 Gift Recovery Exits Are Unavailable While the Active Note Tree Is Full	33
Conclusion	35
Appendix	36
Automated Soundness Analysis	36
Verdicts	37
Issue Classification	40

Summary

Type	Privacy	Total Issues	26 (26 resolved)
Timeline	From 2026-07-29 To 2026-08-17	Critical Severity Issues	1 (1 resolved)
Languages	Solidity Go	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	6 (6 resolved)
		Notes & Additional Information	14 (14 resolved)
		Client Reported Issues	4 (4 resolved)

Scope

OpenZeppelin performed a differential audit of the [sunnyside-io/privacy-boost-backend](#) repository with base commit [cb878677](#) and head commit [17ea3f7d](#). This covered all changes from the "Protocol Extensions" [pull request](#), which adds five new features to the core PrivacyBoost protocol as well as relevant optimizations and refactoring of the existing code.

In scope were the following files:

```
.
├── contracts
│   ├── src
│   │   ├── gateway
│   │   │   └── ExternalCallGateway.sol
│   │   ├── hash
│   │   │   ├── AuthPoseidon.sol
│   │   │   └── Poseidon2T4.sol
│   │   ├── lib
│   │   │   ├── LibAuthZeroHashes.sol
│   │   │   ├── LibDeposit.sol
│   │   │   ├── LibDigest.sol
│   │   │   ├── LibEpoch.sol
│   │   │   ├── LibForced.sol
│   │   │   ├── LibGateway.sol
│   │   │   ├── LibGift.sol
│   │   │   ├── LibPoolShared.sol
│   │   │   ├── LibPortal.sol
│   │   │   └── LibPublicInputs.sol
│   │   ├── verifier
│   │   │   ├── Groth16GiftClaimVerifier.sol
│   │   │   └── Groth16PortalDepositVerifier.sol
│   │   ├── AuthRegistry.sol
│   │   ├── PortalDelegate.sol
│   │   └── PrivacyBoost.sol
└── frontend
    ├── constants.go
    ├── deposit_epoch_circuit.go
    ├── deposit_portal_circuit.go
    ├── epoch_circuit.go
    ├── forced_withdraw_circuit.go
    ├── frontier_batch.go
    ├── gift_claim_circuit.go
    └── helpers.go
```

Updates

Update 1: In addition to the Protocol Extensions pull request, we audited [pull request 1294](#). This change increases the spend approval batch capacity to 256 and the corresponding Merkle tree depth to 8, allowing for Safe multisignature wallets to approve more shielded transfers in one transaction. In scope were the following files:

```
.
├── contracts
│   └── src
│       ├── hash
│       │   └── AuthPoseidon.sol
│       ├── interfaces
│       │   └── Constants.sol
│       └── AuthRegistry.sol
└── frontend
    ├── constants.go
    ├── epoch_circuit.go
    └── forced_withdraw_circuit.go
```

The in-scope files actually changed by PR 1294 were [Constants.sol](#) and [constants.go](#). The rest are downstream consumers of these constants, which were audited at the sites where they are invoked. No issues were found.

Update 2: In addition, we reviewed pull requests [1380](#), [1396](#), and [1398](#), covering fixes to issues surfaced in Sunnyside Labs' internal security review. The scope is identical to the original scope, except for the additional helper circuit in [frontend/spendable_npk.go](#). The final audited commit at the end of the fix review is [1651e47](#).

Update: All resolutions and the final state of the audited codebase mentioned in this report are contained at commit [d4c20ae](#).

System Overview

Privacy Boost is a UTXO-based shielded transfer protocol for EVM chains. A user's balance is a set of notes, each committed as a Poseidon hash in an append-only Merkle tree held on-chain. Spending a note requires a zero-knowledge proof of ownership, publishes a nullifier that prevents the note from being spent twice, and appends new note commitments for the recipient and any change. An observer sees that some notes were consumed and some were created, with no way to connect the two.

Transactions are batched into epochs. A relay aggregates many user operations into a single Groth16 proof, verified once on-chain, which amortizes the verification cost across every participant in the batch. Proof generation for the convenient path happens server-side inside a Trusted Execution Environment (TEE), so users do not run a prover locally. Crucially, the TEE is not trusted with spending authority. A user can always exit through a forced withdrawal using only a client-side proof, without the relay's cooperation.

Spending authority derives from an EdDSA key registered in a separate [AuthRegistry](#) contract, whose leaves live in their own Merkle tree. Registration is authorized by an ECDSA or ERC-1271 signature from the account's owner address, which keeps a user's existing wallet as the root of account control.

Audited Changes

The Protocol Extensions pull request is comprised of the following changes.

Block Number-Based Auth Root Expiry

The previous pool copied authorization roots into a round-indexed snapshot schedule that a relay had to populate. This design required a relay to have snapshotted before a forced withdrawal could proceed, introducing a dependence on relay behavior in the forced withdrawal system. In order to ensure the independence of the forced withdrawal path, that machinery is removed. Routine proofs now cite an authorization root directly, and the auth registry records the block at which each superseded root retired. A root then stays admissible for a configured number of blocks after it is superseded.

Portal Deposit Addresses

The new protocol extensions add support for "portal" accounts, a reusable public address that accepts ordinary ERC-20 transfers and later credits the value to a hidden shielded account. The sender needs no Privacy Boost software and never learns the recipient. The portal address is the user's own EOA, delegated under EIP-7702 to a shared implementation that stores a one-way commitment to the recipient in the EOA's own account storage. Sweeping is permissionless, so the owner need not be online for each deposit, and the party who triggers the sweep learns nothing about who is credited within the shielded pool.

Claimable Gift Transfers

A sender can shield funds to a wallet that has not yet registered with the protocol. The recipient later re-mints the value as their own note, either privately through the relay or through a permissionless public exit. A sender-chosen deadline allows a refund if the gift is never claimed, and claim and refund share a single nullifier so the two paths are mutually exclusive.

External Call Gateway

A withdrawal destination may be registered as a gateway, in which case the withdrawal executes a signed external call rather than a plain transfer. The on-chain executor enforces a target-and-selector policy with explicit input and output token accounting, and settles a failed call into a pending deposit rather than losing the funds.

Batched Note-Tree Append

All three appending circuits replaced their per-leaf note insertion with a shared gadget that compacts the batch and reduces it level by level, changing the cost of inserting K commitments into a depth- D tree from roughly $K \cdot D$ permutations to roughly $K + D$. This is not one of the five changes above; it is a substitution beneath all of them, and it is why the otherwise-unchanged deposit circuit is in scope.

Pool Refactor

The pool contract `PrivacyBoost` was decomposed: large function bodies moved out of `PrivacyBoost.sol` into external libraries invoked by `delegatecall`, in order to stay within the EIP-170 bytecode limit.

Security Model and Trust Assumptions

Chain and Token Assumptions

The protocol assumes the underlying chain provides EIP-7702 support where portal accounts are deployed. Registered tokens are assumed to behave as standard ERC-20s; because the portal and gateway paths measure balance deltas rather than accepting a declared amount, rebasing and fee-on-transfer tokens are unsupported.

Off-Chain Components

Several properties that the reviewed contracts and circuits rely on are supplied by components outside this audit's scope.

Trusted Execution Environment

The TEE holds viewing keys with which it is able to decrypt transaction metadata and input these as secret witnesses to the batched transaction circuits. However, it does not hold any account owner's EdDSA key, on-chain approval, or gift-bearer secret. As such, the TEE is entrusted with privacy and availability of the convenient path, but *not* spending authorization. A compromised TEE could represent a loss of user privacy, but not user funds.

The forced withdrawal escape hatch protects users from issues with TEE availability, either malicious (censorship) or benign (server downtime). It allows a user to withdraw their funds from the pool up to eight notes per withdrawal, after the delay of a cancellation window and subject to the withdrawal fee, which is protocol-wide and capped at 10%.

Discovery Registry

While integrity of the amount and recipient of a portal deposit is enforced on-chain, portal recipient *anonymity* is enforced entirely off-chain. A server-side discovery registry maps each public portal address to the two secret values that open its commitment. Anyone able to read this registry in plaintext could link every portal to its owner. The registry is encrypted at rest under a key held only inside the TEE, and is retained indefinitely by design, since a reusable portal can receive a deposit at any time.

Owner Package and Verifier

A spend approval is represented on-chain as a sorted list of opaque commitments; nothing in the calldata reveals the recipient, amount, or change that produced them. The contracts and circuits together guarantee that the spend which executes is exactly the one the approved commitment represents. What they cannot establish is that the commitment corresponds to the transaction the quorum intended to approve. That link is established off-chain by a set of *owner packages*, each containing the transaction data for one batch item. The owners recompute the commitments from the owner packages using independent verification software. Verification must be all-or-nothing, since a single unverifiable item hidden under the batch root is an unreviewed spend authorization. The correctness and independent distribution of the verifier is a security assumption.

Randomness

Several secrets must be freshly pseudo-randomly sampled with sufficient entropy. Naturally, this cannot be verified on chain, and validation consists only of checking that the pseudorandom number is nonzero and within the appropriate range. This applies to * the user-generated note randomness, which hides the connection between the user's widely-shared master public key and their notes, * the operator-generated display salt, which hides the connection between a spend approval and the spend it authorizes, and * the claim secret, which is a bearer credential in secret-bearer claimable gift transfers.

Insufficient entropy in the first two results in a loss of anonymity, while insufficient entropy in the third risks loss of funds.

Auth Expiry System Migration

Migration inputs are admin-supplied and trusted. The one-time hydration that carries legacy authorization roots into the new anchor scheme validates only that a supplied root is non-zero and belongs to an existing tree; it cannot verify that the root was ever genuinely current. A root seeded in error is admissible for a full staleness window, during which a retired key could authorize a spend. This is a trust assumption rather than an availability concern.

Groth16 Trusted Setup

Security of the Groth16 verifiers assumes that at least one participant in each circuit's ceremony destroyed their toxic waste. Any change to a circuit necessitates the derivation of a new verifying key by trusted setup ceremony, so this trust assumption applies to each new or altered circuit in scope.

Privacy Boundary

Amounts, recipients, and note ownership are hidden, but several properties are public by construction.

Each epoch type has its own entrypoint and event, and each event carries the leaf count before and after, so an observer can label every leaf in the note tree by the kind of operation that created it. Entry metadata is public. However, since nullifiers, as opaque hashes, are indistinguishable by type, and reveal nothing about the leaf they consume, the anonymity set for a spend remains every unspent leaf in the note tree regardless of its origin.

Within a routine epoch, nullifiers and output commitments are supplied grouped by transfer slot. Which inputs pair with which outputs inside a single transfer is therefore disclosed, even though the transfer's owner is not. Batch occupancy is consequently a privacy parameter: a batch containing one transfer offers that user no cover from the batch itself.

The portal path publishes the credited note's value, which equals the swept amount visible in the sweep transaction. The recipient stays hidden; the amount does not.

Two exit paths identify the spending account on-chain. Forced withdrawal takes the spender's account identifier in the clear, which the documentation describes as an accepted trade-off for an emergency path. The wallet-recipient public gift exit publishes the exact authorization leaf, and because every component of that leaf's preimage appears in registration events, the leaf can be reversed to the owning account and from there to the owner's address.

Privileged Roles

The following roles are defined by the reviewed contracts. Their responsibilities are described as encoded in the code; the addresses holding them, and the governance around those addresses, were outside the scope of this audit.

- **Pool Owner:** Sets protocol fees and treasury, updates verifier references, configures portal sweep fees and token thresholds, and appoints the gateway route manager.
- **Pool Proxy Admin:** Upgrades the pool implementation and executes associated initialization, including the one-time gateway route manager migration.
- **Verifier Owner:** Registers and replaces verifying keys for supported circuit shapes, including shapes affected by circuit regeneration.
- **Pool Operator:** Maintains the pool relay allowlist and is appointed by the pool owner.
- **Pool Relay:** Aggregates operations into epochs and submits proofs; it assembles batches but does not authorize their contents.

- **Gateway Route Manager:** Designates withdrawal destinations that settle through External Call Gateways rather than plain transfers.
- **Gateway Policy Owner:** Maintains allowed target, selector, and token combinations, appoints the guardian, and can lift a pause.
- **Gateway Guardian:** Can pause gateway execution and remove individual policies.
- **Authorization Registry Owner:** Appoints the registry operator and may perform the one-time root-freshness anchor migration with the proxy admin.
- **Authorization Registry Proxy Admin:** Upgrades the registry implementation and may perform the one-time anchor migration with the registry owner.
- **Authorization Registry Operator:** Maintains the registry relay allowlist and is appointed by the registry owner.
- **Authorization Registry Relay:** Submits owner-signed key registrations, rotations, and revocations; spend approvals must still be called by the account owner.
- **Account Owner:** Controls authorization keys, approval-only spend approvals, and cancellation of pending forced withdrawals for the account.
- **Token Registry Owner:** Maintains the mapping from compact token identifiers to ERC-20 addresses; the registry contract itself was outside the audit scope.

Automated Soundness Analysis

Alongside manual review, we ran an automated determinacy pass over the five settlement circuits. Each is compiled to its rank-1 constraint system and put through two independent engines, a symbolic closure procedure that needs no witness and a witness-dependent cross-check, to answer one question: does the constraint system leave the prover any free choice that can reach an output? Such freedom is *under-constraint*, and it is the defect class least likely to be caught by reading because a circuit can compute the intended result on every honest witness and still admit a second, dishonest one. The pass was not part of the agreed scope; we ran it as additional assurance.

It identified the critical issue independently of our internal AI tools, reporting C-01 as UNDER-CONSTRAINED on all three circuits that verify an EdDSA signature, each with a concrete counterexample in which the same circuit and the same scalar admit two different outputs. The defect is invisible under honest witnesses and surfaces only when the relevant hint is forced onto the degenerate branch available to a malicious prover. Re-running against the fixed dependency version turns all three verdicts around, establishing the upgrade as the remedy rather than merely a newer release.

It also cleared both deposit circuits outright, with 100% of their internal wires determined without assumptions at production parameters rather than the smaller default test shape,

including the shared frontier-append gadget compiled into them. On the other three circuits, the residual uncertainty is confined to the EdDSA gadget alone, one contiguous span of wires in each circuit with nothing failing outside it. This reduces the caveat to a single named assumption rather than a general disclaimer.

Determinacy is necessary for soundness but not sufficient. It shows that the prover has no free choice, not that the determined computation expresses the intended relation. Two findings in this report are of that second kind, and no determinacy analysis would have surfaced either. Appendix A gives the method, per-circuit results, the assumption underlying three verdicts and what would be required to discharge it, and the limits of the technique.

Critical Severity

C-01 EdDSA Signature Verification Is Satisfiable With a Forged Signature

Three in-scope circuits authorize through the shared `VerifyEdDSAIf` gadget: wallet-bound recipient claims in `GiftClaimCircuit`, newly introduced by this pull request, and, when `useApproval == 0`, forced withdrawals in `ForcedWithdrawCircuit` and auth-key-mode transfers in `EpochCircuit`. Gift refunds, secret-bearer claims, and Safe approval paths do not use EdDSA and are unaffected. `computeEdDSAVerifyArtifacts` checks $S \cdot B8 == R8 + h \cdot (8A)$ using gnark v0.14.0's `curve.ScalarMul` for both `h \cdot (8A)` and `S \cdot B8`. Internally, `scalarMulFakeGLV` accepts a prover-supplied hinted result and verifies a decomposition $s1 + s2 \cdot s \equiv 0 \pmod{\ell}$ plus $[s1]P + [s2]Q = (0,1)$, but never constrains $s2 \neq 0$. A malicious prover can therefore set $s1 = s2 = 0$, satisfying the checks for any scalar and leaving the scalar-multiplication output unconstrained. The output is also not checked on-curve, since the [on-curve assertions](#) cover only `A` and `R8`. Both scalar multiplications are consequently prover-controlled, so `left == right` can be satisfied with arbitrary equal points and `VerifyEdDSAIf` accepts a forged signature for any registered public key without its secret key. Neither this verifier nor the gnark v0.14.0 pin is changed by the pull request; the pull request adds a new circuit that reaches the existing defect.

Settlement trusts only the Groth16 proof. In permissionless `LibGift.publicGiftExit`, a forged recipient-claim proof can consume the gift nullifier and transfer the ERC-20 funds to the proof-bound destination. The remaining recipient witness is reconstructible from public data: the auth leaf and account ID derive from `AuthRegistry.register` calldata, while the leaf index and Merkle path are exposed through `getAuthKeyInfo` and public node storage. An attacker therefore needs only the gift opening contained in the gift ciphertext and claim link to claim or publicly exit a wallet-bound gift without controlling that wallet, effectively degrading wallet-bound gifts to bearer instruments. Forced withdrawals and transfers still require the note's nullifying key, so there the flaw bypasses only the auth-key policy layer, including which registered key may authorize the spend and its expiry, rather than note ownership. `LibBabyJubJub.isValidPublicKey` does not mitigate the issue because it only ensures `8A` is not the identity, while the forgery works for ordinary prime-order keys. A two-witness counterexample confirmed that the same circuit and scalar admit distinct satisfying assignments with different scalar-multiplication outputs.

Consider upgrading gnark to [v0.15.0](#), which fixes the gadget in [PR #1751](#) by enforcing `s2 != 0` with `api.AssertIsEqual(api.IsZero(s2), 0)` and handling a genuine zero scalar separately. Alternatively, replace `curve.ScalarMul` with an implementation that fully constrains its output, including curve membership, prime-order subgroup membership, and a non-degenerate decomposition. Apply the fix to every `curve.ScalarMul` call site, not only EdDSA, and add invalid-signature negative tests for every `VerifyEdDSAIf` path, specifically exercising `useApproval == 0`. Upgrading gnark changes the compiled R1CS, so it also requires a new trusted setup and redeployed Groth16 verifiers.

Update: Resolved in [pull request #1343](#).

Medium Severity

M-01 Fixed Gateway Gas Reserve Does Not Scale With Remaining Slots and Can Revert an Entire Epoch

`_executeGateway` retains a fixed [500,000 gas](#) before invoking an untrusted gateway, but `processGatewayWithdrawals` calls it internally in a loop, so the reserve protects only the current slot and does not account for the work remaining in the epoch. A gateway target can consume nearly all forwarded gas through `ExternalCallGateway._callNoReturndataCopy`, after which fallback recording, including `LibDigest.computeGatewayDepositData`, can consume most of the reserve. Subsequent slots may then run out of gas in an internal call, which cannot be caught and reverts the entire epoch through `PrivacyBoost.submitEpoch`.

A PoC confirms that one gas-starved slot can settle through the fallback path while two can revert the complete epoch over a broad range of transaction gas limits, including limits close to the normal cost of a healthy two-withdrawal epoch. The relay's pre-flight simulation does not necessarily detect this because `simulateGatewayWithdrawals` runs as an uncapped `eth_call`, allowing the same batch to succeed in simulation but revert when submitted with a bounded transaction gas limit. No funds are directly lost, but a single adversarial or unexpectedly expensive gateway execution can prevent settlement for every withdrawal batched alongside it until the relay re-batches the epoch.

Consider scaling the retained gas with the work still outstanding, for example `slotsRemaining * worstCaseRecordingCost`, and enforcing a minimum `gasleft()`

before entering the fallback recording path so a starved slot fails locally rather than exhausting the batch. Bounding the number of gateway slots per epoch or moving expensive post-call work such as Poseidon derivation out of the settlement transaction would further reduce the risk. The relay should also simulate using its intended submission gas limit, with a regression test covering deliberately gas-consuming gateway targets.

Update: Resolved in [pull request #1344](#).

Low Severity

L-01 Disallowed ERC-20 Recipient Can Revert Entire Portal Deposit Epoch

When a relayer submits a deposit epoch, the pool [transfers](#) the fee amount of the given ERC-20 token to the sweeper for each (sweeper, token) pair in a loop. The iterations of this loop are not enclosed in a try-catch block, so a revert on any of the transfers reverts the entire portal deposit epoch submission. The comments [state](#) that malicious contracts cannot revert a batch since the ERC-20 is being called, rather than the recipient. However, ERC-20 transfers can revert on attempting a transfer to a recipient that is blacklisted or not whitelisted. A contract that is not an allowed recipient for a given ERC-20 can therefore [permissionlessly](#) become a sweeper and revert an entire portal deposit epoch.

The utility of this as an intentional attack vector is low: either the protocol must register an ERC-20 with a whitelist, the attacker must already control a blacklisted account, or the attacker must try to get themselves blacklisted as a recipient for an ERC-20 that is already registered with PrivacyBoost's token registry. Additionally, this can be mitigated operationally by having the TEE exclude the disallowed recipient from the next epoch submission, though this may raise censorship concerns. The most realistic scenario is that an honest but disallowed recipient reverts one portal deposit epoch.

Consider enclosing the fee payment loop in a try-catch block to tolerate failed receipt by disallowed recipients of an ERC-20 token.

Update: Resolved in [pull request #1346](#).

L-02 Auth Key Mutations Fail for Account Owners that Delegate Under EIP-7702

In the `AuthRegistry` contract, the `register`, `rotate`, and `revoke` functions each authorize the caller and then unconditionally verify an owner signature through `_verifyOwnerSig`, so the signature is required even when the account owner calls directly. That helper delegates to `SignatureChecker.isValidSignatureNow`, which branches on `signer.code.length`. A codeless signer is verified with ECDSA, while a signer with code is verified with ERC-1271 only, without an ECDSA fallback. This partitions owners into plain EOAs and ERC-1271 contracts, but EIP-7702 introduces a third category: namely, an externally owned account that carries a delegation indicator as code while remaining controlled by its private key. When such an owner submits a conventional ECDSA signature, the ERC-1271 dispatch fails, the `expectedOwner.code.length == 0` guard does not trigger because the account has code, and execution falls through to the ERC-7739 nested-signature parser, which rejects the signature with `InvalidSignatureLength`.

The protocol distributes a delegate that exhibits this behavior: `PortalDelegate` implements `portalBinding`, `initializePortal`, `initializePortalWithSig`, `sweep`, and `withdraw`, but no `isValidSignature`, `fallback`, or `receive`, so the ERC-1271 staticcall reverts and resolves to false. An account owner who delegates their EOA to `PortalDelegate` or any other delegate lacking ERC-1271 support cannot register, rotate, or revoke auth keys, losing access to the key-rotation and revocation paths that exist for compromise recovery. Account creation and the spend-approval functions are unaffected, since they authorize on `msg.sender` alone. Additionally, the condition is reversible since an EIP-7702 delegation can be revoked at any time, restoring the ECDSA branch. Consider adding an `ECDSA.recover(rawDigest, sig) == expectedOwner` check to `_verifyOwnerSig` after the ERC-1271 attempt and before the ERC-7739 parse. This would restore signature verification for every delegated EOA regardless of its delegate and cannot weaken security for genuine contract owners, as no private key exists for a contract's own address. Alternatively, or in addition, consider implementing ERC-1271 in `PortalDelegate`, noting that this addresses the issue only for this delegate, requires portal owners to re-delegate to the new implementation address, and makes portal accounts signature-capable toward any ERC-1271 consumer.

Update: Resolved in [pull request #1347](#).

L-03 Secret-Bearer Claims Are Not Constrained to Public-Payout Settlement

The gift-claim circuit selects bearer mode through the private witness `SecretMode` and selects settlement through the public `BranchSelectors`, where `1` denotes private mint and `0` public payout. No constraint links these values, so `SecretMode = 1` and `BranchSelectors = 1` is satisfiable, allowing a bearer claim to settle as a private mint to a claimant-chosen `TargetMPK`. This contradicts the specification requirement that bearer claims be available only through a public exit. The stated external enforcement cannot provide this guarantee because `SecretMode` is private, while `buildGiftClaimInputs` currently hardcodes `BranchSelectors = 1` for every slot.

The impact is loss of the intended observability guarantee rather than additional authority over funds. A party that knows the bearer secret can already redirect the gift to an arbitrary destination, but a public payout exposes the token, amount, and destination on-chain, whereas a private mint hides these values behind a commitment. As a result, a bearer-secret holder can bypass the specified public traceability requirement. Exposure is currently limited because bearer mode is disabled by default and audit-gated.

Consider enforcing the invariant inside the circuit, for example with `AssertIsZeroIf(api, secretClaim, c.Pub.BranchSelectors[c0])`, so bearer claims can only use the public-payout branch. Alternatively, making the mode public would allow contract-level enforcement but would change the public-input layout. The specification should also be updated because the current statement that this invariant is enforced outside the circuit is not achievable while `SecretMode` remains private.

Update: Resolved in [pull request #1348](#).

L-04 An EOA Delegating to `PortalDelegate` Under EIP-7702 Can No Longer Receive ETH

Under [EIP-7702](#), a delegated EOA executes the delegate's code for ordinary calls, including plain ETH transfers. `PortalDelegate` defines neither `receive()` nor a payable `fallback()`, so after delegation, plain ETH transfers and other empty-calldata calls revert. A PoC using the same EIP-7702 cheatcode as the project's `Portal.t.sol` confirms this for `.call`, `transfer`, and zero-value empty-calldata calls.

The impact is loss of normal ETH-receiving functionality for the delegated account. The protocol does not appear to depend on the portal EOA receiving ETH, and its sweep path

handles ERC-20s, so this is primarily an availability issue rather than loss of protocol funds. However, any external use that expects normal EOA behavior will fail for as long as the delegation remains active.

Consider adding `receive() external payable {}` to preserve the account's pre-delegation ETH-receiving behavior.

Update: Resolved in [pull request #1349](#).

L-05 Forced-Withdrawal Fees Stranded upon Treasury Unset

Forced withdrawals snapshot the fee rate at request time, [storing](#) the then-current `withdrawFeeBps` into the request record and [binding](#) it to the proof digest, so a later rate change cannot alter an already-accepted request. The treasury address, by contrast, is not snapshotted: instead, it is [read](#) from live pool storage at execution. Settlement always subtracts the snapshotted fee from the gross amount and pays only the net to the recipient, while [gating](#) the fee transfer itself on `treasury != address(0)`. Because `_setFees` [accepts](#) zero unconditionally and `setTreasury` [permits](#) the zero address once the global rate is zero, calling `setFees(0)` and then `setTreasury(address(0))` leaves every request that was accepted while the fee was non-zero settling as `gross - fee` with no fee recipient.

The deducted amount remains in the pool as unattributed balance. Since no generic surplus-withdrawal path exists, it is permanently unrecoverable. The two-step sequence unsetting the treasury is also not purely hypothetical, but a plausible operator response to a treasury address that has become untransferable for a given token.

Consider treating the fee as zero at settlement whenever no recipient is configured, for example by computing `uint96 feeAmount = treasury == address(0) ? 0 : uint96((uint256(grossAmount) * feeBps) / BASIS_POINTS)`, so that the full gross amount is paid out when the fee cannot be delivered.

Update: Resolved in [pull request #1350](#).

L-06 Fee-Note Ciphertext Metadata Is Not Integrity-Protected

Note metadata travels in calldata: a `Transfer` carries a shared `viewingKey` and `teeWrapKey`, and each `Output` carries `receiverWrapKey` and `ct0..ct3`. For user transfers it is authenticated: `computeTransferDigest` hashes `abi.encode(..., nullifiers, outputs, viewingKey, teeWrapKey)`, which covers every field of every `Output`, and the digest becomes the `ApproveDigestHi/Lo` proof inputs that the spender signs — so tampering invalidates the proof. The fee note inherits none of that protection, because no party signs it: `_buildFeeCommitments` reads only `feeTransfer.outputs[i].commitment`, the epoch's fee-related public inputs are just `FeeNPK` and `FeeCommitmentsOut`, and `feeTransfer` is referenced nowhere else beyond a length check. An allowlisted relay can therefore submit a valid proof with correct fee commitments while replacing `feeTransfer.viewingKey`, `feeTransfer.teeWrapKey` and each fee output's ciphertext with arbitrary bytes.

Those fields are the fee recipient's only discovery channel, so affected notes become unspendable rather than merely undiscoverable. The indexer decodes `feeTransfer` from `submitEpoch` calldata and decrypts each fee note with exactly the unauthenticated values, and unlike shielded deposits — whose ciphertext it re-fetches from the `ShieldRequested` event — fee notes have no event-sourced fallback. Reconstruction without that ciphertext is infeasible, since spending requires the note's value and that is 96 bits. The commitment is still appended to the tree and the pool still holds the backing ERC-20, so the value is stuck rather than destroyed and pool solvency is unaffected; the loss falls on the fee recipient. Exposure is presently limited because `submitEpoch` is relay-gated and the operator both builds epochs and owns the treasury, but the issue should be re-rated upward if relays become permissionless or fees are directed to a third party, since the party able to corrupt the metadata and the party harmed by it would then differ.

Consider binding a hash of the fee metadata into the epoch's public inputs and constraining it in-circuit, so `feeTransfer.viewingKey`, `feeTransfer.teeWrapKey` and each fee output's ciphertext fields are integrity-protected alongside `FeeCommitmentsOut` — mirroring the coverage the transfer digest already provides for user outputs. Alternatively, if calldata is not intended as the fee recipient's authoritative recovery channel, consider delivering fee openings over an authenticated channel instead and documenting that on-chain fee metadata is advisory.

Update: Resolved in [pull request #1354](#).

Notes & Additional Information

N-01 Poseidon Hash Function Accepts Non-Canonical Arity

The `hashUpTo5` sponge helper documents that `len` must be between two and five but does not enforce this bound. Its `branch structure` treats values above five like `len == 4` for input absorption, while the sponge commits `len` only through `mulmod(len, TWO_POW_64, PRIME)`. Consequently, lengths congruent modulo the field prime can alias valid lengths. In particular, `hashUpTo5(PRIME + 5, a0...a3, X)` equals `hashUpTo5(5, a0...a3, 0)` for every `X`, because the committed length is congruent to five while the `len == 5` branch is not taken, so `a4` is never absorbed.

The function is externally reachable with an unconstrained length through `AuthPoseidon.hash`, so external integrators can receive a digest whose apparent arity does not match the values actually absorbed. This is particularly relevant to five-input structures such as `registration leaves`, where a non-canonical length can produce the same digest while silently omitting the final input. No current protocol behavior is affected because all internal call sites use literal lengths from two through five.

Consider enforcing `2 <= len && len <= 5` directly in `hashUpTo5WithConstants`, near the `spongeInit` call, so the documented precondition is enforced at the underlying entry point and covers both internal and external callers. This bound also eliminates the modular aliases of valid lengths, so a separate check that the length is canonical is not required.

Update: Resolved in [pull request #1355](#).

N-02 Non-Canonical Signature Scalar $S = \ell$ Accepted

`computeEdDSAVerifyArtifacts` enforces $S < \ell$ (where ℓ is the BabyJubJub subgroup order) via Circom's `CompConstant`, but sets the comparison constant to ℓ . Since `CompConstant(ct)` returns `1` only when the input is strictly greater than `ct`, enforcing `sTooLargeBit == 0` permits $S \leq \ell$ instead of $S < \ell$, so the non-canonical value $S = \ell$ ($\equiv 0 \bmod \ell$) passes. No practical exploit was identified — values above ℓ are still rejected,

and nullifiers are derived independently of the signature (`Poseidon2T4(domain, NullifyingKey, InputNoteLeafIndex)`).

Consider setting the constant to `ℓ - 1` (the correct bound for this strict-greater-than comparator), or using gnark's `cmp.IsLess(api, s, ℓ)` and enforcing the result equals `1`.

Update: Resolved in [pull request #1356](#).

N-03 Portal Note Randomness Lacks Domain Separation by Deployment

The per-portal hiding factor is `derived` as `blind = KDF(viewing_secret, E)`, which does not bind the chain ID nor the pool address. Because `E` is an externally owned account whose address is identical on every EVM chain, the same recipient operating the same portal against two deployments derives the same blind, and therefore the same owner binding `H`, the same `noteRnd`, and the same NPK; if the counter, tokenId, and credited amount coincide, then the same `output commitment` is derived as well.

This carries no risk to funds and no additional disclosure. However, since identical commitments and bindings can legitimately exist on multiple deployments, any off-chain consumer that keys portal notes by commitment or by `H` without scoping to `(chainId, pool)` will observe duplicates.

Consider deriving the blind as `KDF(viewing_secret, chainId, pool, E)` so that every portal-derived value is domain-separated by deployment by construction.

Update: Resolved in [pull request #1357](#).

N-04 Forced Withdrawal Recipient Public Input Is Not Range-Constrained

In the forced withdrawal circuit, the public input `WithdrawalTo` is declared but never range-constrained. This is not currently exploitable because the contract constructs the public-input vector: `LibForced` derives both the approval digest through `computeForcedWithdrawalDigest` and `withdrawal.to` from the same `Withdrawal` calldata struct before passing them to `buildForcedWithdrawalInputs`, so the recipient cannot disagree with the value committed by the digest.

The issue is therefore one of consistency and circuit-level validation. The adjacent public inputs `NIn`, `WithdrawalAmount` and `WithdrawalTokenID` are [range-constrained](#), while `WithdrawalTo` alone is not, requiring reviewers to rely on contract-side construction to establish its validity. Adding the missing check costs roughly 160 constraints.

Consider adding `AssertIsNBits(api, c.Pub.WithdrawalTo, AddressBits)` with a new `AddressBits = 160` constant alongside the existing `CountBits`, `TokenIDBits` and `AmountBits`, matching the contract's `uint256(uint160(withdrawalTo))` encoding. If the omission is an intentional optimization, consider documenting it explicitly.

Update: Resolved in [pull request #1358](#).

N-05 `AuthContext` Field Layout Is Split Across Two Layers, and the Circuit Reads as Though Half of It Were Padding

`ForcedWithdrawCircuit.verifyAuth` decomposes the 128-bit public `AuthContext` input and [constrains only part of its layout](#): `authExpiry` at `[0,64)`, `useApproval` at bit `64`, `version == 1` at `[100,108)`, and bits `[108,128)` asserted zero. Bits `[65,100)` are neither read nor constrained, but they are not spare capacity — `LibForced._decodeAuthContext` reads `leafIndex` from `[65,85)` and `treeNumber` from `[85,100)` using [its own shift and mask constants](#), widths that exactly match their domains: a leaf index in a depth-20 auth tree, and a tree number bounded by `MAX_AUTH_TREE_NUMBER = 32767`.

No malleability results, because every field is pinned by one layer or the other: `authExpiry` is hashed into the auth leaf the circuit equates to the public `AuthLeaf`, `useApproval` selects the leaf kind in-circuit and the registry getter on-chain, `version` and the high bits are asserted on both sides, and `_resolveLiveAuth` requires the decoded `leafIndex` and `treeNumber` to equal the registry's record for the supplied `authId`. The concern is legibility: a reader of the circuit alone sees `version` pinned and the top bits zeroed, and will reasonably conclude that `[65,100)` is reserved padding — an inference that makes zeroing those bits look like safe hardening when it would break every forced withdrawal from a non-zero leaf index or tree number. The registry cross-check is their only enforcement, and nothing on the circuit side records that.

Consider documenting the full 128-bit layout at `verifyAuth`, noting that `[65,100)` carries `leafIndex` and `treeNumber` whose validation is delegated to `LibForced._resolveLiveAuth`. Decomposing both fields in-circuit even though they are

unused there — `api.FromBinary(contextBits[65:85]...)` and `api.FromBinary(contextBits[85:100]...)` — would assert the layout in one place at negligible cost, and holding the shift and mask boundaries in a single shared source rather than as [named constants in LibForced.sol](#) and bare literals in the circuit would keep the two from drifting apart.

Update: Resolved in [pull request #1359](#).

N-06 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions. The smart contracts in scope were each found to be lacking a security contact.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: Resolved in [pull request #1360](#).

N-07 Use `calldata` Instead of `memory`

When dealing with the parameters of `external` functions, it is more gas-efficient to read their arguments directly from `calldata` instead of storing them to `memory`; `calldata` is a read-only region of memory that contains the arguments of incoming `external` function calls. This makes using `calldata` as the data location for such parameters cheaper and more efficient compared to `memory`. Thus, using `calldata` in such situations will generally save gas and improve the performance of a smart contract.

Throughout the codebase, multiple instances where function parameters should use `calldata` instead of `memory` were identified:

- In `LibDigest.sol`, the `output` parameter of `computeGiftClaimDigest`

- In `LibEpoch.sol`, the `config` parameter of `verifyAndSpend`
- In `LibPublicInputs.sol`, all `memory` parameters of `buildEpochInputs`, `buildDepositInputs`, `buildPortalDepositInputs`, and `buildGiftClaimInputs`

Consider using `calldata` as the data location for the parameters of `external` functions to optimize gas usage.

Update: Resolved in [pull request #1361](#).

N-08 Unnecessary `unchecked` Blocks

Starting from Solidity [version 0.8.22](#), the compiler automatically optimizes arithmetic increments in `for` loops by removing overflow checks.

Throughout the codebase, multiple instances of unnecessary `unchecked` blocks were identified:

- In `AuthPoseidon.hashSpendApprovalBatch`, the `unchecked{++i;}` and `unchecked{++level;}` blocks.
- In `AuthPoseidon.hashAuthPath`, the `unchecked{++level;}` block.
- In `LibEpoch._computeTransferDigests`, the `unchecked{++gatewayCursor;}` and `unchecked{++withdrawalCursor;}` blocks.
- In `LibGateway.processGatewayWithdrawals`, the `unchecked{++gatewayCursor;}` block.
- In `LibGateway.processGatewayWithdrawalsForSim`, the `unchecked{++gatewayCursor;}` block.

Consider removing these unnecessary `unchecked` blocks to improve the overall clarity of the codebase.

Update: Resolved in [pull request #1362](#).

N-09 Gift Claim Circuit Inactive Slot Constraints are Unreachable

In `LibPublicInputs`, the `buildGiftClaimInputs` function derives the per-slot array dimension and the value of the `NClaims` public input from the same expression, `giftNullifiers.length`, `emitting` as the active-slot count the same value it used to size

all seven per-slot columns. Since `submitGiftClaimEpoch` requires `giftNullifiers.length == batchSize` and `dispatches` on a verifying key registered for exactly that batch size, the circuit's compiled `MaxGiftClaims` is always equal to `NClaims`. Therefore `claimActive[c0]` holds for every slot, the inactive-slot branch is dead, and `NClaims` is fixed by the verifier through the public-input vector length. The gift claim circuit is the only one in which the number of claims and slots are necessarily the same: the other four circuits all support variable-length inputs padded to the number of slots.

Consider passing the active claim count to `buildGiftClaimInputs` as its own parameter, with the per-slot arrays padded to the registered batch size and `giftNullifiers.length` bounded by rather than equal to it, mirroring `buildPortalDepositInputs`. The circuit's `claimActive` gating and the witness builder's `giftClaimZeroSlot` padding already implement the prover side, so this makes existing constraints load-bearing rather than adding any, and removes the need for a separate trusted setup per exact batch size.

Update: Resolved in [pull request #1363](#).

N-10 `cancelDeposit` Does Not Follow CEI Pattern

In `LibDeposit.cancelDeposit`, the depositor is `refunded` before the entry in the `pendingDeposits` mapping is `cleared`, in violation of the Checks-Effects-Interactions (CEI) pattern. Currently there is no concrete reentrancy vulnerability arising from this instance, since the importing `PrivacyBoost` contract `applies` the `nonReentrant` modifier to `cancelDeposit`. However, the fact that the only safeguard preventing a potentially draining reentrancy attack lies in a separate file creates unnecessary maintenance hazard.

To observe Solidity best practices and reduce maintenance hazard, consider reordering the operations of `cancelDeposit` to follow the CEI pattern.

Update: Resolved in [pull request #1364](#).

N-11 Non-Descriptive Names

Several values carry single-letter or heavily abbreviated names drawn from the design documents rather than names describing their role in the code. In Solidity the uppercase forms

are additionally misleading, since that casing conventionally signals a contract, type, or constant rather than a parameter.

- The portal deposit address `E`: appears in the external signature `requestPortalDeposit(address E, uint16 _tokenId)` and throughout `LibPortal`, so it reaches generated bindings and documentation. The same value is already named `portal` in the pending-deposit struct field, in the `PortalInitialized` event, and in the interface `NatSpec`, and `PortalAddresses` in the portal deposit circuit. Consider renaming this to `portal`.
- The portal owner binding `H`: used for the namespaced storage field, the event parameter, and the `initializePortal` and `initializePortalWithSig` parameters. This includes the `H` field in the EIP-712 `PortalBind` type string, which is also what a wallet displays when an owner signs a relayed registration. The same value is named `recipientBindH` in `LibPortal` and `LibDigest`, and `RecipientBindHs` in the circuit and `LibPublicInputs`. Consider renaming this data field to `recipientBinding`.
- The gift recipient slot `W`: holds two different things depending on the claim branch — a 20-byte recipient wallet address for a wallet-bound gift, and `low160(Poseidon(domainGiftSecret, claimSecret))` for a secret-bearer gift, where no wallet is involved. A single letter obscures that the slot is deliberately polymorphic; every neighbouring witness field uses a descriptive name. Consider renaming this witness to `RecipientSlot`.
- The guardian address `g`: a public setter parameter, so like `E` and `H` it is part of the external interface. The state variable it assigns to is already named `guardian`. Consider renaming this state variable to `guardian`.
- The call policy `p`: used as the parameter to both `setCallPolicy` and `_setCallPolicy`, and as the local in `_validateCall`, where its fields are read six times in five lines. Consider renaming this parameter to `policy`.
- The withdrawal record `w`: the loop and parameter name for a `Withdrawal calldata` across four functions in `LibGateway` and one in `LibEpoch`, including in the routing branch that decides whether the destination is a gateway. Consider renaming to `withdrawal`.
- The pending deposit record `pd`: denotes a `PendingDeposit` in `LibGateway` and `LibDeposit`, but a `PortalPendingDeposit` in `LibPortal` — two distinct structs sharing one abbreviation across the rescue and credit paths. Consider renaming to `pendingDeposit` and `portalPendingDeposit`, respectively.
- In the `_callPolicy` state variable of the `ExternalCallGateway` contract, the mapping does not have any named parameters. In other locations the key to this mapping is called a `policyKey`. Additionally, mapping names are usually plural,

referring the set of values. Consider naming the keys and values in the mapping at its definition, such as `mapping(bytes32 policyKey => CallPolicy policy)` `private _callPolicies`.

To make the codebase more self-documenting, consider renaming the above objects to the more descriptive forms the codebase uses elsewhere, and adding named keys and values to the `_policyKey` mapping.

Update: Resolved in [pull request #1365](#).

N-12 Magic Numbers in the Code

Throughout the codebase, there are occurrences of literal values with unexplained meanings. The instances below fall into three groups, each representing a single named quantity that is currently repeated as a bare literal across multiple files. In each case the value is load-bearing, so a future change requires locating and updating every site by hand.

- Maximum authorization-tree depth (20), and the zero-hash table sized at depth + 1 (21). The validated upper bound, the sibling and parent array widths, and the zero-hash table length all encode the same quantity. Instances are in `AuthRegistry.sol` twice on line [122](#) and once each on lines [629](#), [631](#), and [646](#); and `AuthPoseidon.sol` on lines [64](#), [67](#), and [69](#). Consider declaring a constant `MAX_AUTH_TREE_DEPTH = 20`, with the arrays declared as `uint256[MAX_AUTH_TREE_DEPTH]` and the zero-hash table as `uint256[MAX_AUTH_TREE_DEPTH + 1]`.
- Maximum note-tree depth (24), and the zero-hash table sized at depth + 1 (25). The same pattern for the note tree, where the 25 appears in a function whose loop bound is the unrelated `SPEND_APPROVAL_BATCH_DEPTH`, leaving the reader with no local indication of what the value refers to. Instances are in `PrivacyBoost.sol` twice on line [289](#), and in `AuthPoseidon.sol` once on line [43](#). Consider declaring the constant `MAX_NOTE_TREE_DEPTH = 24`, with the zero-hash table declared as `uint256[MAX_NOTE_TREE_DEPTH + 1]`.
- ERC-7739 signature suffix length (66). This value is the combined width of the trailing app domain separator (32 bytes), contents hash (32 bytes), and contents-description length field (2 bytes), and it governs both the length check and the offset used by the subsequent assembly reads. Nothing in the surrounding code states the decomposition. Instances are in `AuthRegistry.sol` on lines [549](#) and [551](#). Consider declaring the constant `ERC7739_SUFFIX_LENGTH = 66`, matching the naming of the `CONTENTS_DESCRIPTION_LENGTH` constant already declared in the same contract.

To improve the readability of the codebase and maintainability of the codebase, consider defining and using constant variables instead of using literals. Solidity permits constant expressions in fixed-size array declarations, so the depth + 1 relationship between each bound and its zero-hash table can be expressed directly rather than encoded as an independent literal. Note that `LibAuthZeroHashes.get` and `LibZeroHashes.get` return `uint256[21]` and `uint256[25]` respectively, and would need to be updated in the same change for the definitions to remain consistent.

Update: Resolved in [pull request #1366](#).

N-13 Inconsistent Integer Base Within Contracts

In assembly, memory offsets can be declared using decimals or hexadecimals, but it is important to use a consistent notation to avoid confusion and to make it easy to understand the purpose and usage of these sizes.

Throughout the codebase, there are multiple contracts that have a inconsistency in the way memory offsets are declared:

- The `AuthRegistry` contract in `AuthRegistry.sol`.
 - Memory offsets declared using decimals:
 - `add(appStart, 32)`
 - Memory offsets declared using hexadecimals:
 - `add(sig, 0x20)`
 - `add(sig, 0x20)`
- The `Poseidon2T4` contract in `Poseidon2T4.sol`.
 - Memory offsets declared using decimals:
 - `add(o, 32)`
 - `add(o, 64)`
 - Memory offsets declared using hexadecimals:
 - `mstore(0x40, rc)`
 - `mstore(0x40, rc)`

Consider using a consistent notation for expressing memory offsets throughout the codebase.

Update: Resolved in [pull request #1367](#).

N-14 Missing or Incomplete Docstrings

In several contracts, instances of missing or incomplete docstrings were identified.

- `ExternalCallGateway.sol` is missing docstrings throughout.
- `AuthPoseidon.sol` is missing docstrings throughout.
- In `LibForced.sol`, the external `requestForcedWithdrawalSuffix` function is missing documentation.
- In `LibGateway.sol`, the external functions `processGatewayWithdrawalsForSim` and `processOneGatewaySlotForSim` are undocumented.
- In `LibDigest.sol`, the external function `computeGatewayDepositData` is missing documentation for its input and return parameters.
- In `PrivacyBoost.sol`, the external functions `rescueGatewayDeposit` and `simulateGatewayWithdrawalsInner` are missing documentation of their input and return parameters.

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #1471](#).

Client Reported

CR-01 Gift Refunds Authorize Safe-Owned Spends on an Operator-Held Key

The [sender-refund branch](#) of the gift claim circuit authorizes a spend on knowledge of `senderNk` alone, constraining only that `senderMPK` derives from the account ID and nullifying key, that the opened `refundField` matches the value hashed into `giftNPK`, and that the refund deadline has elapsed. No auth registry membership is proven. The [stated rationale](#) is that "only the original sender can produce a refund proof", since the quorum already approved the funding, and this assumption rests on `senderNk` being known solely to the account owner. That premise holds for EOA-owned accounts, where the nullifying key derives from a master seed produced by the owner's own wallet signature. It cannot hold for Safe-owned approval-only accounts, since a Safe is a contract with no private key and cannot

produce such a signature. An approval-only account's nullifying key is therefore [generated and retained by the operator SDK](#). This goes against the Spend Approval system's stated trust model, in which the quorum approves an exact transaction and does not consider the operator as a trusted third party.

The exposure extends to the server, since proving is server-side and the nullifying key transits to the server. Both the operator and the server can thus construct a refund proof for any Safe-funded gift, while the payout destination is bound only by [ComputeGiftExitDigest](#) and chosen freely at proving time. Refunds [force CommitmentsOut to zero](#) so that the pool [skips its leaf-liveness check](#). Once the [minimum 129,600-block refund delay](#) elapses, either party can therefore move a Safe-funded gift to an arbitrary address with no quorum decision, to a destination the owners never saw when they approved the funding, and with no revocation path available to stop it. Consider requiring the refund branch to prove an authorization leaf in the auth registry, selecting between the sender's registered key leaf and a Safe approval leaf bound to the refund digest, and exposing that leaf in [CommitmentsOut](#) so the pool can reject a revoked key or approval at settlement.

Update: Resolved in [pull request #1268](#).

CR-02 Direct-Key Gift Refunds Do Not Authenticate the Payout Destination

The gift-claim circuit verifies an EdDSA signature in exactly one place, namely [inside assertRecipientBranch](#) and gated on the wallet-bound recipient claim, so the sender-refund branch never reaches it. A direct-key refund proves that the sender nullifying key and the original gift opening [re-derive the value hashed into the gift](#), that a live authorization-key leaf is [a member of an admissible authorization tree](#), and that the refund deadline has elapsed. It deliberately does not prove possession of the private key behind that leaf. The leaf's contents are public registry data and its Merkle path is reconstructible from public storage, so the whole witness can be assembled from the sender nullifying key, the gift opening, and chain state, and [publicGiftExit](#) then pays the destination carried by the proof's public inputs, which nothing on this branch binds to the signing key.

The service and prover architecture handle the sender nullifying material and gift openings, so a compromised service or enclave can construct a valid post-deadline refund to an address of its own choosing without the sender's authorization private key, redirecting each eligible gift whose refund material it holds. The shared gift nullifier prevents a second spend but does not authenticate the first payout destination. Recipient claims verify a signature over the claim digest, Safe approval refunds bind the digest through the approval commitment, and private-

mint refunds are fixed to the sender master public key, so the exposure is confined to direct-key public refunds after the deadline on gifts that remain unclaimed. Consider requiring an in-circuit EdDSA signature over the complete claim digest for every direct-key public refund, with the signing public key constrained to the active authorization leaf so that the chain, pool, gift, token, amount, minimum net amount, destination, and operation domain are all covered by the signature.

Update: Resolved in [pull request #1396](#).

CR-03 A Withdrawal Marker Can Be Reopened as a Normal Note

A withdrawal marker is [Poseidon\(DOMAIN NOTE, uint160\(to\), tokenId, amount\)](#), which has the same domain, arity, and field ordering as a note commitment, which is [Poseidon\(DOMAIN NOTE, npk, tokenId, amount\)](#), with only the second argument differing. The epoch circuit [binds the first output of a withdrawal transfer to the marker](#) through the [ordinary output-commitment relation](#) and then [appends every active output](#), marker included, to the spendable note tree, while the ordinary and [forced](#) spend relations reconstruct an input note with that identical formula. An attacker can therefore fix an account and nullifying key, vary [noteRnd](#), and search for a derived note public key below 2^{160} , then use that value as the withdrawal address. The expected cost is $r / (2^{160} - 1)$, approximately $2^{93.6}$ Poseidon evaluations for the BN254 scalar-field modulus r , and the result is independent of token and amount and reusable across withdrawals.

The first withdrawal transfers the value to that low address and appends a marker whose note opening the attacker knows, and a later spend of the appended leaf pays the same value a second time, so a successful search produces an asset-conservation failure. The first destination is a Poseidon output rather than an address under the attacker's control, so that payout is effectively burned rather than captured: the attacker recovers close to their principal through the second spend and the missing collateral falls on other depositors. The computation is not practical with current resources. It is nonetheless well below the roughly 127-bit collision bound that is the weakest generic attack on a field of this size, which is the standard the construction should meet.

Consider refusing any note public key inside the below- 2^{160} address range in every spend relation, including the ordinary, forced, and gift-claim paths, so that a marker is structurally unopenable as a note. Because this leaves the marker formula untouched it preserves every deployed root and signed withdrawal request, and it also covers markers already present in live trees. If the range test is implemented by decomposing the key into bits, take that

decomposition at the full scalar-field width so the reducedness check is emitted alongside the recomposition constraint: at a narrower width a prover could answer the bit hint with a non-canonical representation that recomposes to the same field element while setting high bits, satisfying the very predicate the test exists to fail. Consider excluding withdrawal markers from the tree append using a mask the contract derives from the already-validated withdrawal slots rather than by a prover-supplied flag, and subtracting excluded markers from the leaf count delta so that the capacity check and the circuit agree. Both changes alter the epoch constraint system, so the affected proving keys, verifying keys, Solidity artifacts, and deployment manifest must be regenerated. Consider also scanning stored notes and outstanding gifts for keys already inside the reserved range before the new relations are enabled, since an honest historical key that happens to land there would become permanently unspendable, and a gift key is derived in a separate domain that an ordinary note scan cannot reach.

Update: Resolved in [pull request #1453](#).

CR-04 Gift Recovery Exits Are Unavailable While the Active Note Tree Is Full

A public gift exit appends no leaf, so `buildGiftExitInputs` describes the tree transition as a no-op and [hardcodes the rollover flag to zero](#), carrying the active root and count through unchanged as the post-state. The gift-claim circuit, however, reads a cleared rollover flag as an assertion that the output tree still has room, [requiring `CountOld` to be strictly below the tree's capacity](#) whenever the flag is not set. Once the active tree reaches capacity the two are contradictory: the count the contract supplies equals the maximum, the rollover flag the contract supplies is zero, and no witness satisfies the constraint. `publicGiftExit` can therefore not process a gift note held in the active tree at all, not because the proof is rejected on chain, but because it cannot be produced. Wallet-recipient claims, sender refunds and secret-bearer exits all settle through this entry point, so the three permissionless recovery modes fail together.

Only the four epoch entry points advance the tree, and each is relay-gated: `submitEpoch`, `submitDepositEpoch`, `submitPortalDepositEpoch` and `submitGiftClaimEpoch`.

A full tree therefore stays full for exactly as long as no relay is submitting, which is the same condition the permissionless exit exists to survive: the recovery path a user is meant to fall back on during a relay or server outage becomes unavailable precisely when that outage coincides with a full tree. No funds are lost, and gifts funded into earlier finalized trees are unaffected, because the funding root is supplied separately from the active-tree state and only

a gift held in the full tree itself would need that tree number carried twice. Recovery for the affected gifts resumes as soon as any epoch rolls the tree.

Consider representing the no-append exit through the circuit's existing rollover branch when the active tree is at capacity, supplying the capacity as `countOld`, a set rollover flag, and the empty-tree root with a zero count as the post-state, which the circuit satisfies without appending anything. Because rollover mode does not bind the active tree root to `countOld` in-circuit, that representation should be admitted only when the caller's root and count match the live full tip, and the caller-attested count should be bounded by the tree capacity. This can be done entirely on the contract side, so unlike the other two findings it needs no change to the constraint system and no artifact rotation.

Update: Resolved in [pull request #1380](#).

Conclusion

The audited changes introduce several new features: replacing the pool's round-based authorization snapshot model with root-keyed freshness anchors, and adding portal deposit addresses built on EIP-7702 delegation, claimable transfers, an external call gateway, and account-owner spend approvals. One critical-severity issue was identified: the EdDSA verification on the auth-key spend paths delegates its scalar multiplications to a gnark v0.14.0 gadget that takes the result from a prover-supplied hint and leaves it unconstrained, allowing a signature to be forged for any registered public key. The defect originates in the proving framework rather than in the protocol's own logic, but it is reachable from three call sites and its remediation changes the compiled constraint system. In general, we found that within each contract and circuit, the code was thoughtfully constructed and validation was thorough: the remaining issues cluster at the boundaries between layers.

The codebase is thoroughly documented. Beyond inline comments that explain why a construction takes the form it does, the team prepared a dedicated documentation package for this review setting out intended behavior, non-goals, trust boundaries and security invariants, and several findings in this report were reachable because that material made the intended semantics explicit enough to audit against. Several further fixes were suggested to improve consistency and clarity, in order to facilitate future audits, integrations and development. We thank the Sunnyside Labs team for being highly responsive throughout the engagement and provided extensive and helpful documentation, and we look forward to supporting their work in the future.

Appendix

Automated Soundness Analysis

Not part of the agreed scope; run as additional assurance.

In short. Both deposit circuits have **100% of their internal wires determined without assumptions** at production parameters. The three circuits that verify an EdDSA signature were reported **UNDER-CONSTRAINED** at the audited gnark version, corresponding to C-01, and **SAFE*** under one named assumption at the fixed version. Determinacy is necessary for soundness but not sufficient; see *Limits*.

Method

Each circuit is compiled to its R1CS and analysed by two independent engines: a symbolic closure procedure whose side conditions are discharged with an SMT solver (cvc5), and a witness-dependent Jacobian cross-check. The question asked is whether the constraint system leaves any **prover-controlled degree of freedom that can affect an output or otherwise influence the verified relation**, given the circuit's designated inputs. Freedom that reaches an output is *under-constraint*, a major class of ZK soundness defect.

Engine A needs no witness and reports each wire as pinned unconditionally, pinned under a guard, or not pinned. *Not pinned* is not the same as free, because closure is incomplete.

Engine B needs a witness and settles the remainder through four gates:

gate	asks
D1	are closure's unconditional pins corroborated by the Jacobian?
D2	is every Jacobian-free wire explained?
C1	is each wire's multiplicity bound exactly 1? (<i>uniqueness proper</i>)
C2	is every free wire inert and not an output?

Verdicts

verdict	meaning
SAFE	no relevant prover-controlled nondeterminism, established outright
SAFE*	the same conclusion, conditional on an explicitly named premise , here Assumption A-2
UNDER-CONSTRAINED	a concrete alternative satisfying assignment was exhibited, demonstrating relevant freedom
reduced coverage	uniqueness could not be established, and no counterexample was found

The last two are deliberately distinct: a C1 failure yields *reduced coverage*, never UNDER-CONSTRAINED, because it may equally reflect incompleteness of the multiplicity calculus on a particular structure. That separation blocks both misreadings: "could not prove uniqueness" is not "vulnerable", and "found no problem" is not "sound".

Weaker still is a *closure-only* SAFE*, where no gate has run and the verdict reduces to "the derivation certificate verified". **No verdict below is closure-only:** a witness was built for each of the five standalone circuits, so engine B and all four gates ran on every one.

Results

Assumption A-2. The gnark EdDSA `ScalarMul` gadget uniquely determines its outputs from its inputs.

circuit	internal wires	determined with no assumption	v0.14.0 (audited commit)	v0.15.0
<code>deposit_epoch_circuit.go</code>	6,262	100%	both engines corroborate; all hint degeneracies rejected	identical across versions
<code>deposit_portal_circuit.go</code>	5,955	100%	as above	identical across versions

circuit	internal wires	determined with no assumption	v0.14.0 (audited commit)	v0.15.0
<code>forced_withdraw_circuit.go</code>	12,339	56.1%	UNDER-CONSTRAINED: 19 wires free and coupled at a degenerate <code>ScalarMul</code> hint	SAFE* under A-2
<code>gift_claim_circuit.go</code>	17,929	69.8%	UNDER-CONSTRAINED: same signature, reproduced independently	SAFE* under A-2
<code>epoch_circuit.go</code>	21,543	74.9%	UNDER-CONSTRAINED: same signature, reproduced independently	SAFE* under A-2
<code>frontier_batch.go</code>	—	—	not a standalone circuit; its constraints compile into the two deposit circuits, so the batched frontier-append logic falls inside their unconditional result	

The deposit circuits need **no assumption at all**, and the verdict holds at production parameters (`DefaultNoteDepth = 20`, `DefaultAuthDepth = 20`, `MaxNoteRootsPerProof = 16`), where they compile to 19,552 and 19,235 constraints rather than the 6,312 and 5,995 of the default test shape.

For the other three, uniqueness failures are **confined entirely to the EdDSA gadget**, one contiguous wire span per circuit, with no failure outside it. That containment is what makes A-2 a narrow premise rather than a blanket caveat.

What A-2 rests on. At v0.15.0 every *enumerated* hint-degeneracy case is unsatisfiable, and three exhaustive searches over specific perturbation classes find no residual freedom, each validated by a positive control. The 254-bit decomposition that permitted non-canonical bit vectors no longer exists; a single-wire substitution scan over all 5,411 residual wires finds

nothing; and coordinated pair negation over a structurally filtered set of admissible candidate pairs finds nothing. A coordinated change across three or more wires is not excluded. **That gap is the entire content of A-2.**

Discharging it means establishing the gadget's output uniqueness from the group law. That argument already exists in kernel-checked form for a different frontend. It composes two proven leaves, bit-decomposition uniqueness and faithfulness of the accumulation ladder to its constraint lowering parametrically in the curve constants. Together they establish the result without solving the ladder directly. It consumes LLZK IR rather than gnark R1CS, so applying it here is a port rather than new proof work.

What the gates rest on

The gates carry every verdict above, so their rules were formalised in Lean rather than left as implementation assumptions. `Multiplicity.lean` proves the degree-to-multiplicity bound C1 applies, which matters because a 0-dimensional Jacobian fibre is *not* uniqueness, and C1 is what closes that gap. `Inertness.residual_benign` does the same for C2. Formalising it exposed that the rule needs **two** hypotheses: a wire must be inert *and* not be an output. The gate had enforced only the first. Until corrected, C2 returned a **false SAFE** on a known published finding used as a positive control. Every SAFE* above depends on that correction.

Lean was applied to the analyser's reasoning rules, not to these circuits; no Lean proof was run against the gnark R1CS.

Limits

- **Determinacy is a structural assurance property, not a characterization of soundness.** Its failure may expose under-constraint, but legitimate witness multiplicity also produces non-determinacy, and determinacy says nothing about whether the determined computation expresses the intended relation. Read every SAFE* as "*no relevant prover-controlled freedom found*", never as "*sound*".
- **C-01 does not appear under honest witnesses.** These checks evaluate at a witness and an ordinary prover runs hints honestly, so the baseline run never enters the degenerate branch and reports clean. The defect surfaces only when the fake-GLV hint is **forced** onto the branch available to a malicious prover, which is how the UNDER-CONSTRAINED verdicts above were obtained.
- **Coverage-bounded.** The adversarial sweep enumerates *hint* degeneracies, which are structurally enumerable; freedom on an unsampled branch remains invisible. Cryptographic gadgets are trusted leaves by design, which is why a defect *inside* one required A-2 to be made explicit.

Reproduction

Circuits were exported from the target-commit worktree via the analyser's gnark frontend and audited with cvc5 on `PATH`. A `poseidon` smoke test reproduced the tool's published 265-constraint milestone, confirming the export pipeline, and derivation certificates verified for every circuit. Runs were repeated at both gnark versions. R1CS artifacts are client-derived and retained in the engagement workspace.

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system. These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.